UNIVERSITY OF ILLINOIS URBANA-CHAMPAIGN

ECE408 - CS483 - CSE408 Spring 2026
Applied Parallel Programming

Lecture 4: Compute Architecture and Scheduling

Volodymyr Kindratenko

1/29/2026

Course Reminders

- Lab 1 is due this Friday
  - Requires Lab 0 to be completed first
  - Easy lab, all code was given in lecture 2, however do not postpone doing it until last minute
  - Also not included in the final grade
- Post your questions on Campuswire, the course staff only replies to questions posted on Campuswire
  - But before posting a question, make sure you try to find the answer yourself first, <u>including asking the course chatbot</u>
- Check office hours schedule, we have in-person and on-line

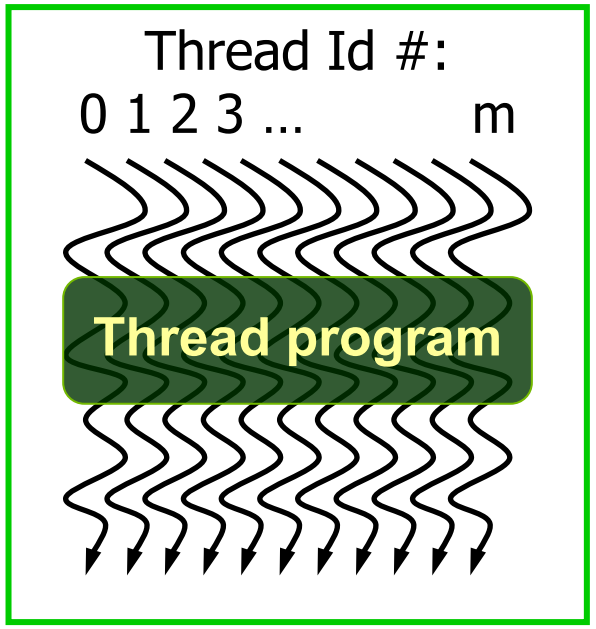
Today's Objectives

- To understand basics of GPU architecture and organization
- To learn the concepts of thread scheduling, latency tolerance, and hardware occupancy

Review: CUDA Execution Model: Thread Blocks

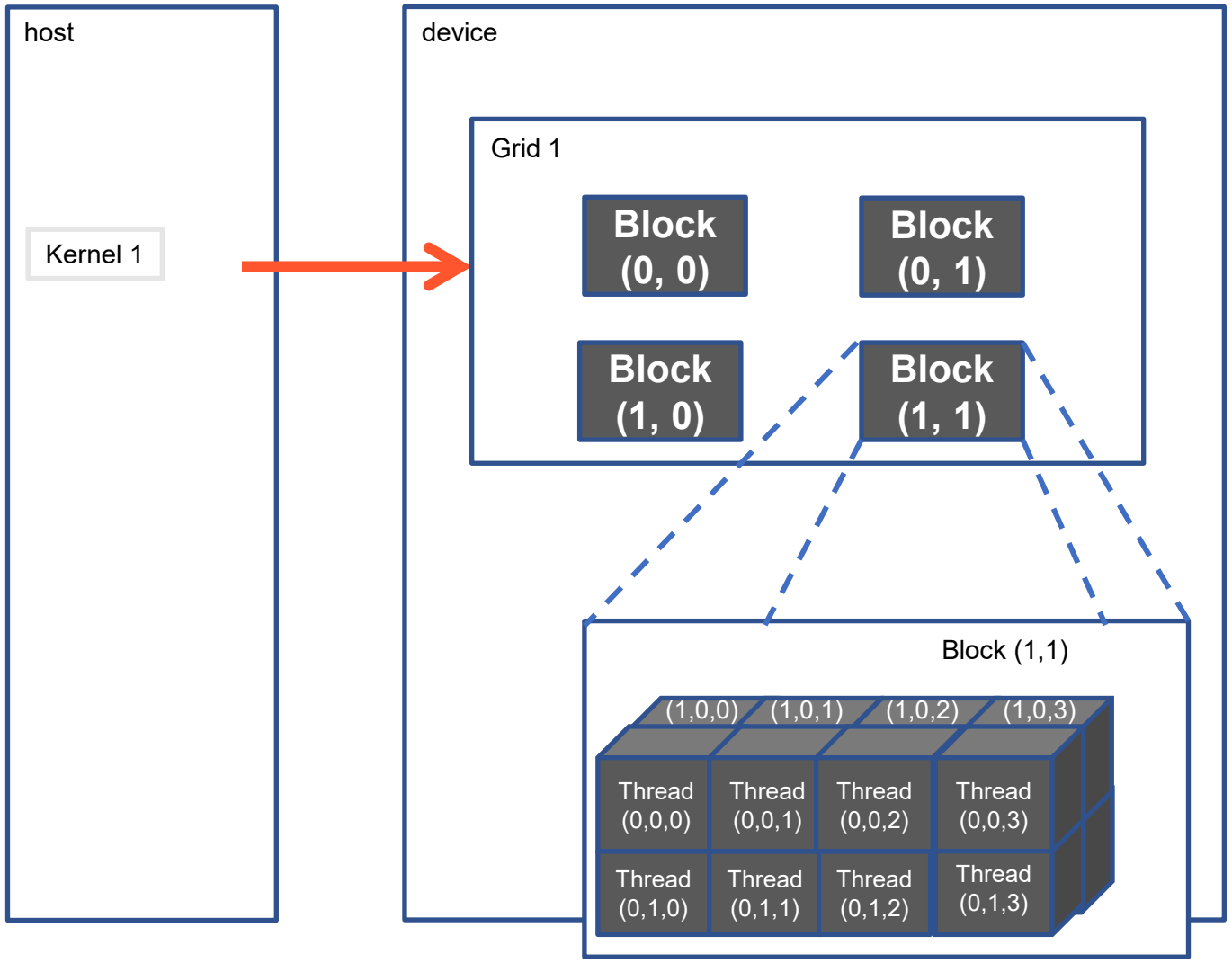
- All threads in a block execute the same kernel program (SPMD)
- Programmer declares block:
  - Block size 1 to 2048 concurrent threads
  - Block shape 1D, 2D, or 3D
- Threads within block have thread index numbers
- Kernel code uses thread index and block index to select work and address shared data
- Threads in the same block share data and synchronize while doing their share of the work
- Threads in different blocks cannot cooperate
- Blocks execute in arbitrary order!

CUDA Thread Block



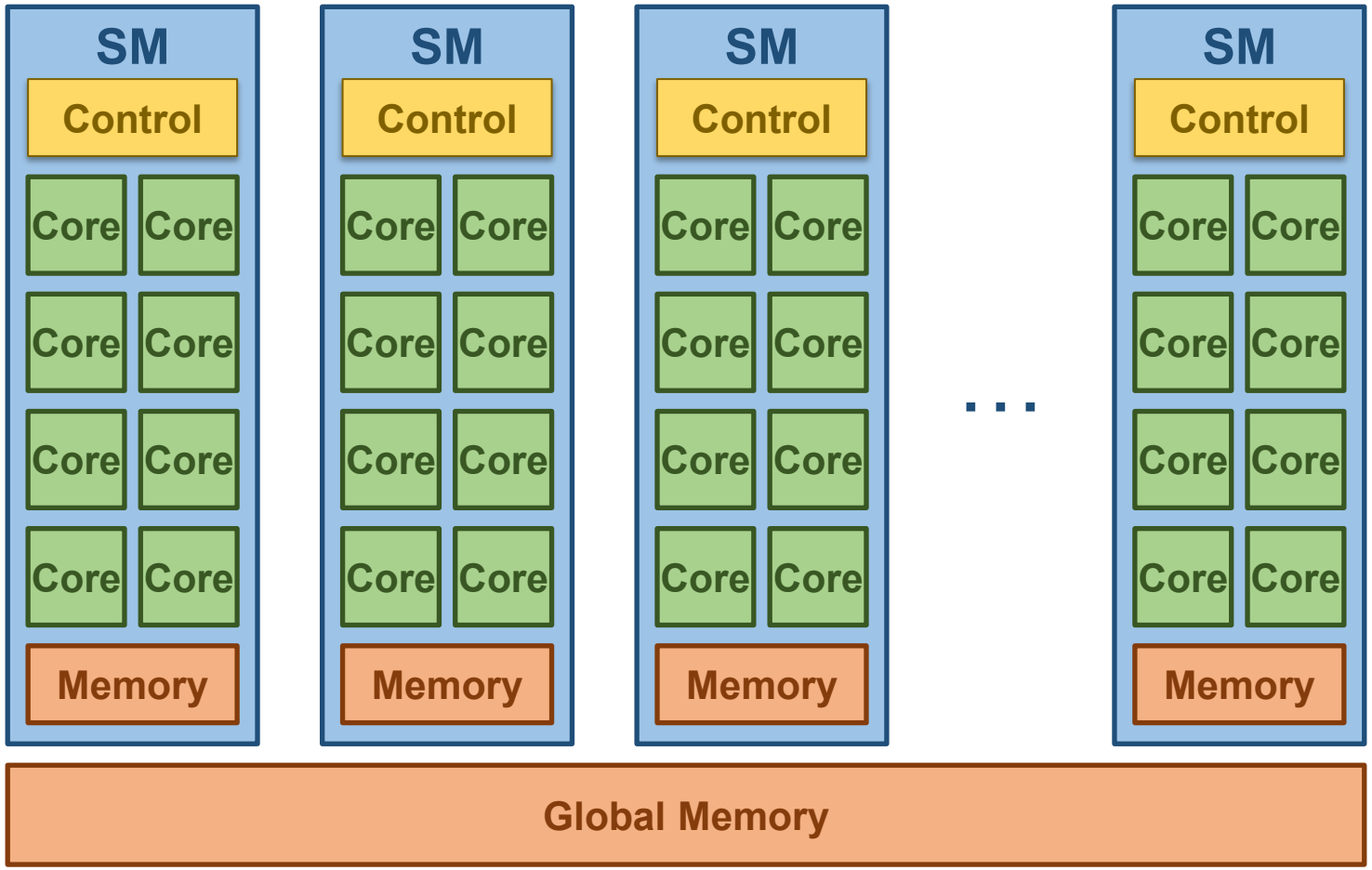
Review: CUDA Thread Grids are Multi-Dimensional

- CUDA supports multidimensional grids (up to 3D)
  - simplifies memory addressing
  - when processing multidimensional data



GPU Architecture

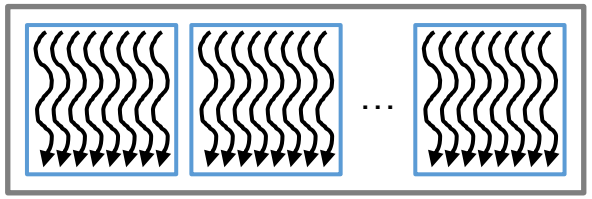
- A GPU consists of multiple Streaming Multiprocessor (SMs), each consisting of multiple cores with shared control and memory
  - e.g., a Hopper H100 GPU has 144 SMs with 128 cores each which totals 18,432 cores



Assigning Blocks to SMs

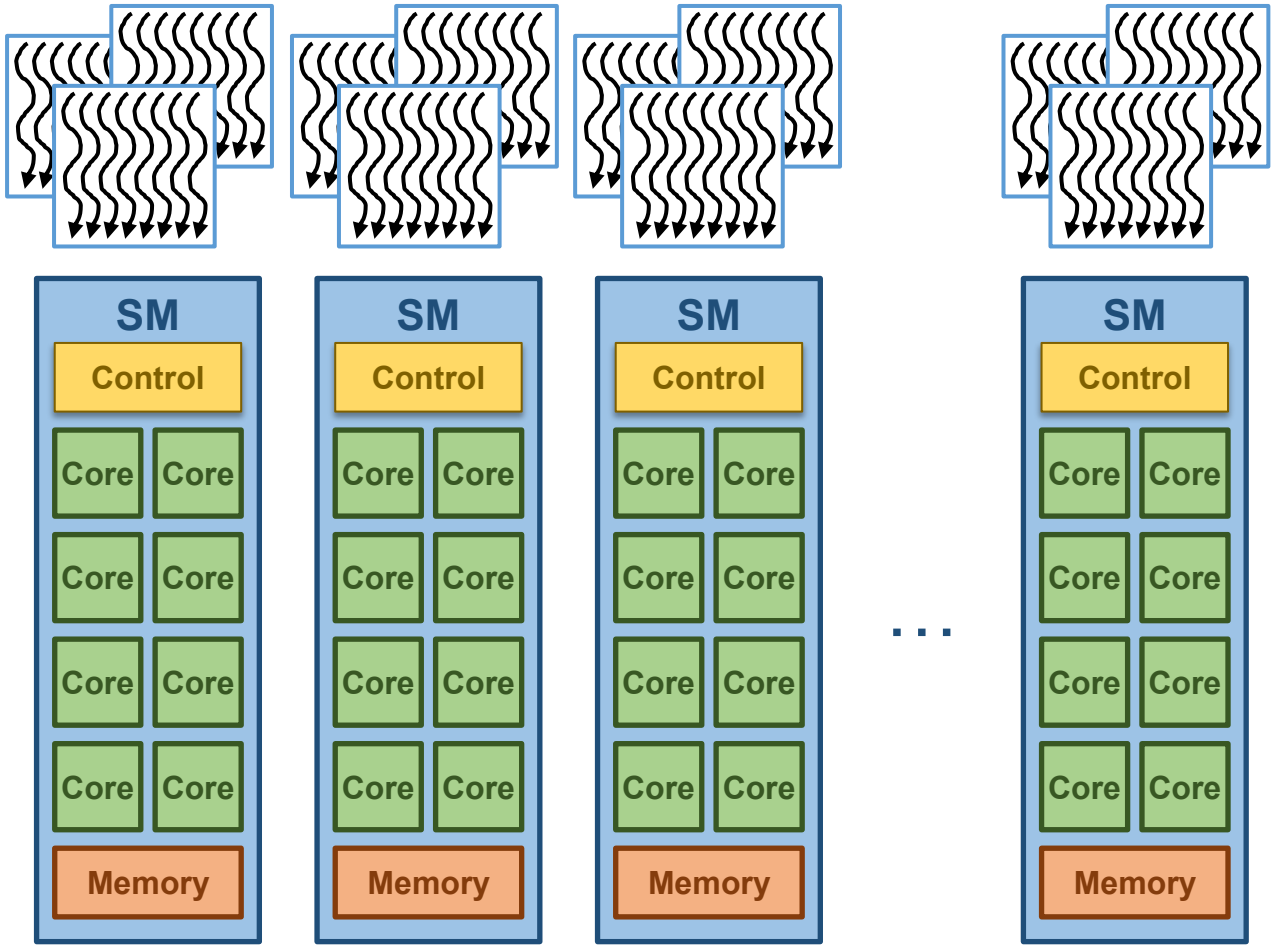
Threads/blocks require resources to execute (e.g., registers, memory, …) so SMs can accommodate a limited number of threads/blocks at once…

… The remaining blocks wait for other blocks to finish before they can be assigned to an SM.



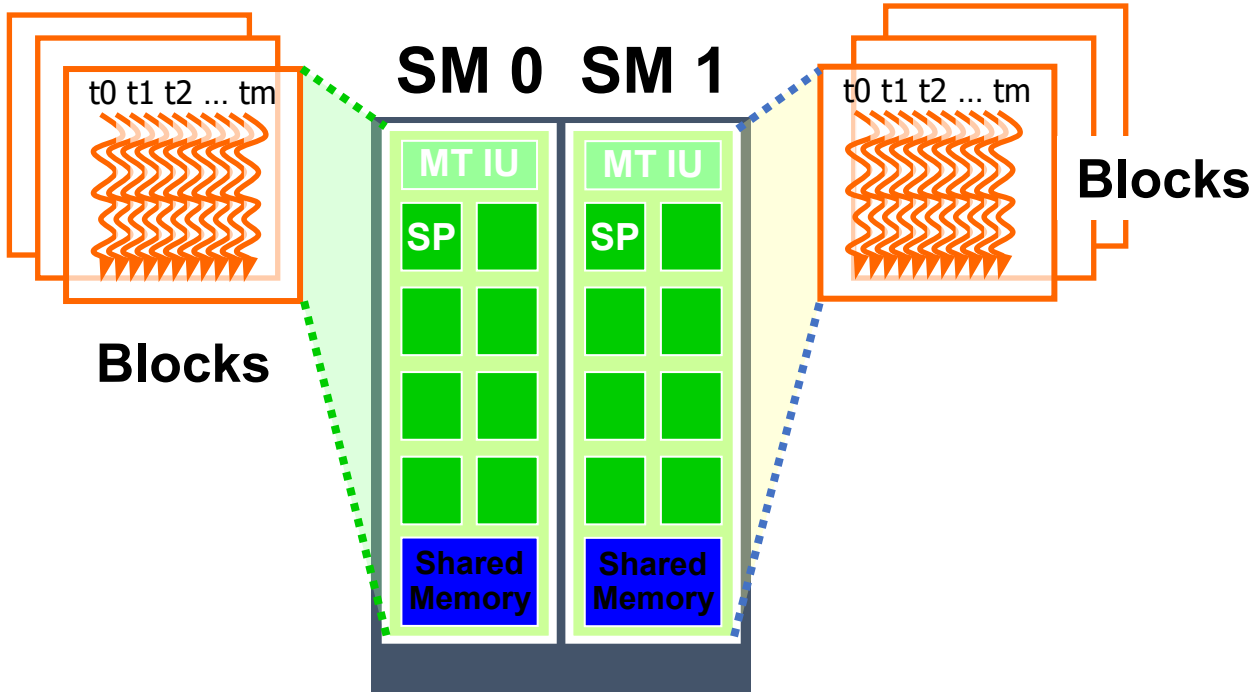
Waiting

Threads are assigned to SMs at block granularity
(all threads in a block are assigned to the same SM)



Executing Thread Blocks

- Threads are assigned to Streaming Multiprocessors in block granularity
  - Up to **32** blocks to each SM
  - SMs can take up to **2048** threads
- Threads run concurrently
  - SM maintains thread/block id #s
  - SM manages/schedules thread execution

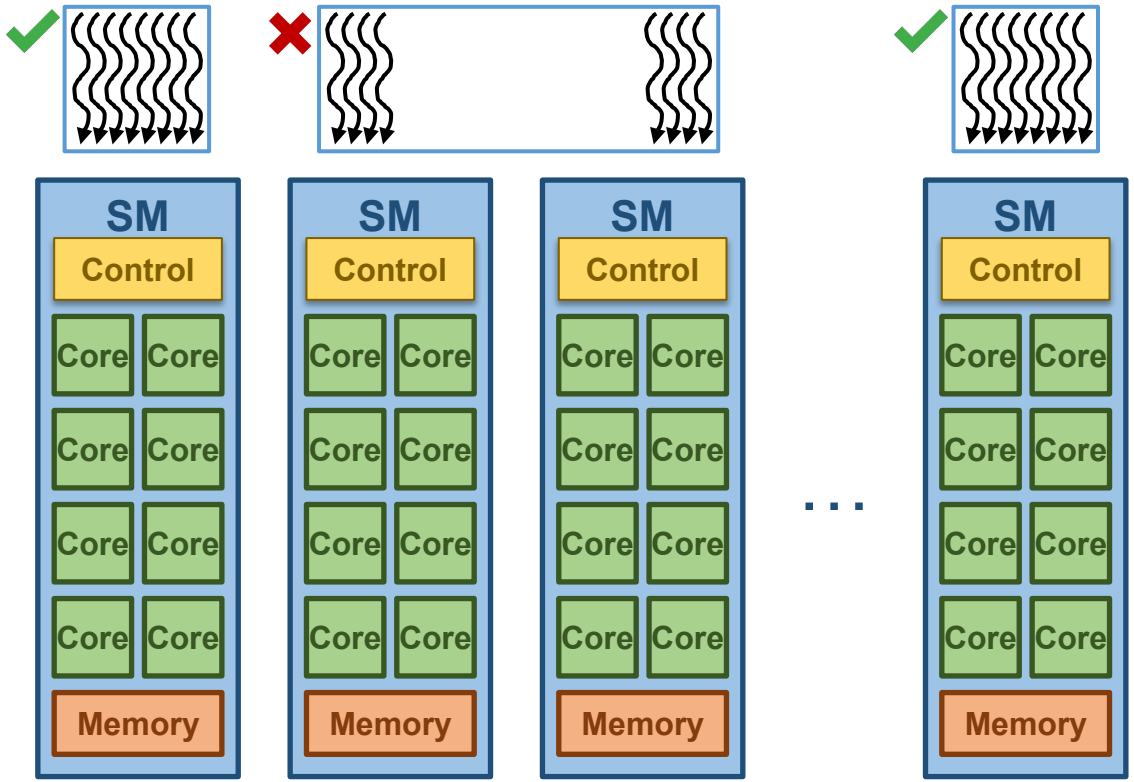


Synchronization within Thread Blocks

- Threads in the same block can collaborate in ways that threads in different blocks cannot:
  - Lightweight barrier synchronization: `__syncthreads()`
    - Wait for all threads in the block to reach the barrier before any thread can proceed
  - Shared memory (discussed later)
    - Access a fast memory that only threads in the same block can access
  - Others (discussed later)

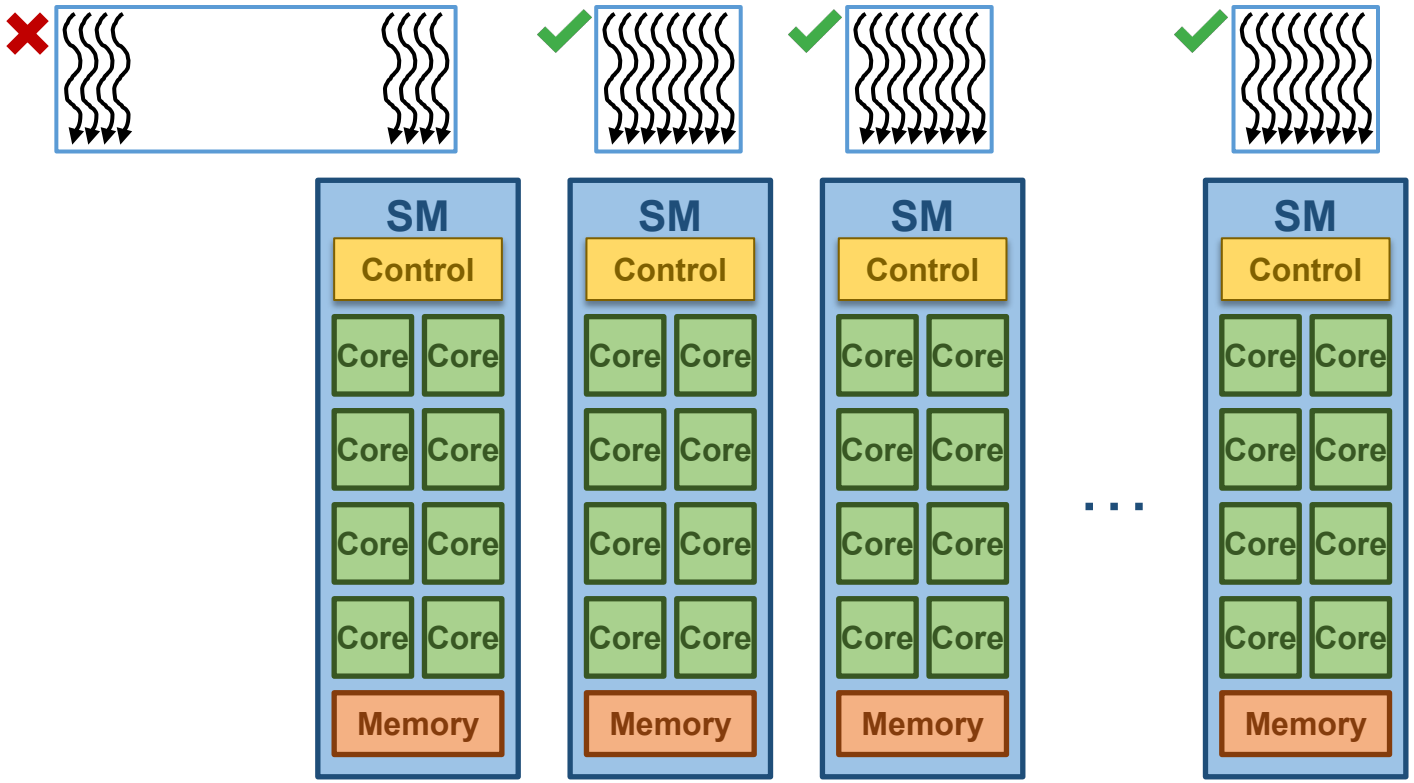
Scheduling Considerations

- Threads in the same block are assigned to the same SM
  - Requiring threads in the same block to be assigned to the same SM makes it easier to support efficient collaboration between them



Scheduling Considerations

- All threads in a block are assigned to an SM simultaneously
  - A block cannot be assigned to an SM until it secures enough resources for all its threads to execute
  - Otherwise, if some threads reach a barrier and others cannot execute, the system could deadlock



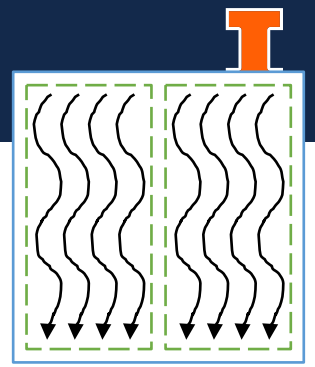
Synchronization across Thread Blocks

- If threads in different blocks do not synchronize with each other
  - Blocks can execute in any order
  - Blocks can execute both in parallel with each other or sequentially with respect to each other
  - Enables **transparent scalability**
    - Same code can run on different devices with different amounts of hardware parallelism
    - Execute blocks sequentially if device has few SMs
    - Execute blocks in parallel if device has many SMs

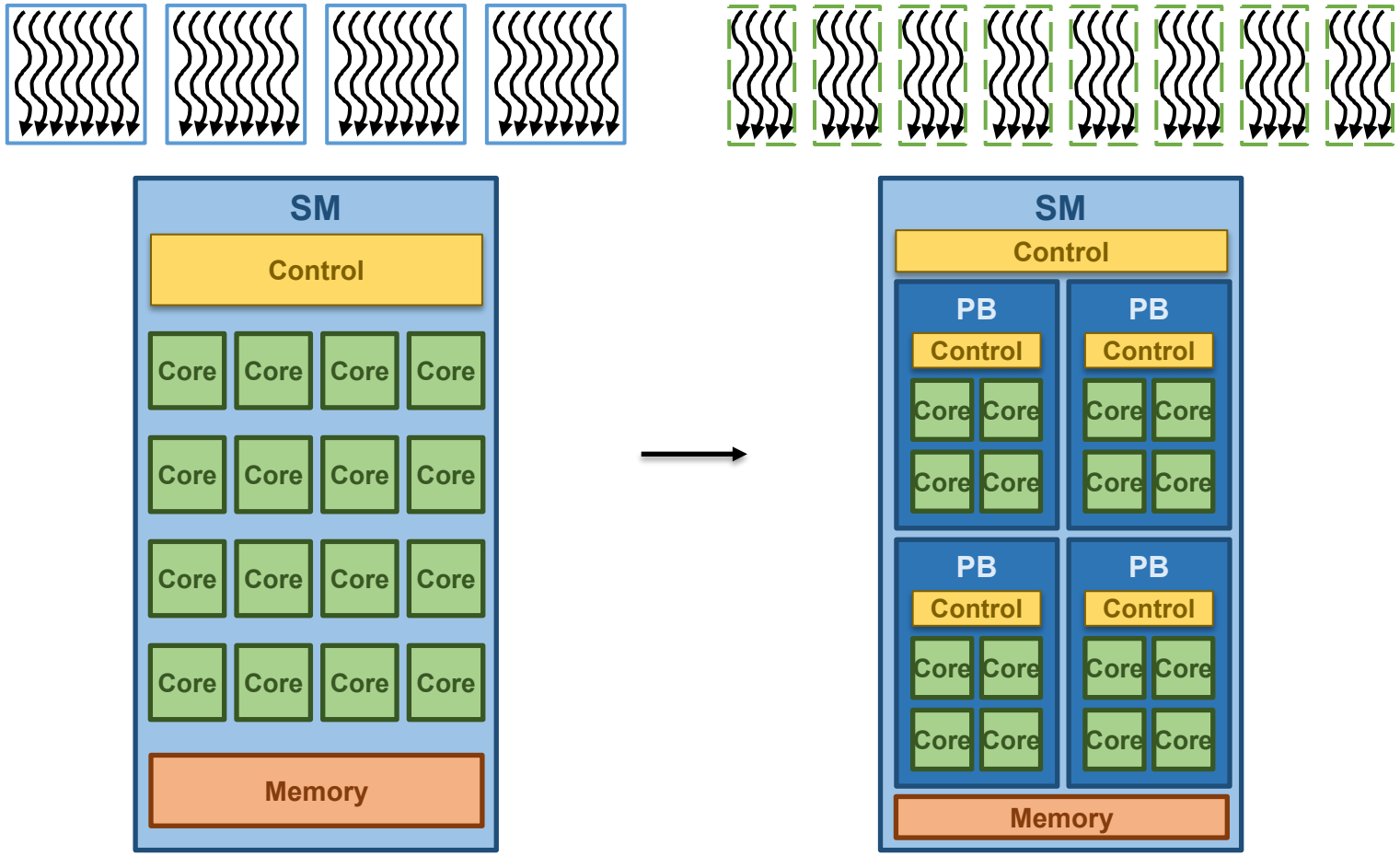
Synchronization across Thread Blocks

- If threads in different blocks to synchronize with each other
  - Deadlock may occur if the synchronizing blocks are not scheduled simultaneously
  - **Cooperative groups** (covered later) allows barrier synchronization across clusters of thread blocks, or across the entire grid by limiting the number of blocks to guarantee that all blocks are executing simultaneously
  - Other techniques (covered later) allow **unidirectional synchronization** by ensuring that the producer block is scheduled before the consumer block

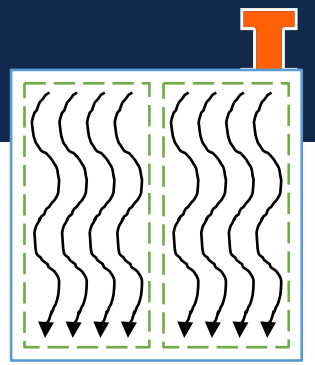
SM Scheduling



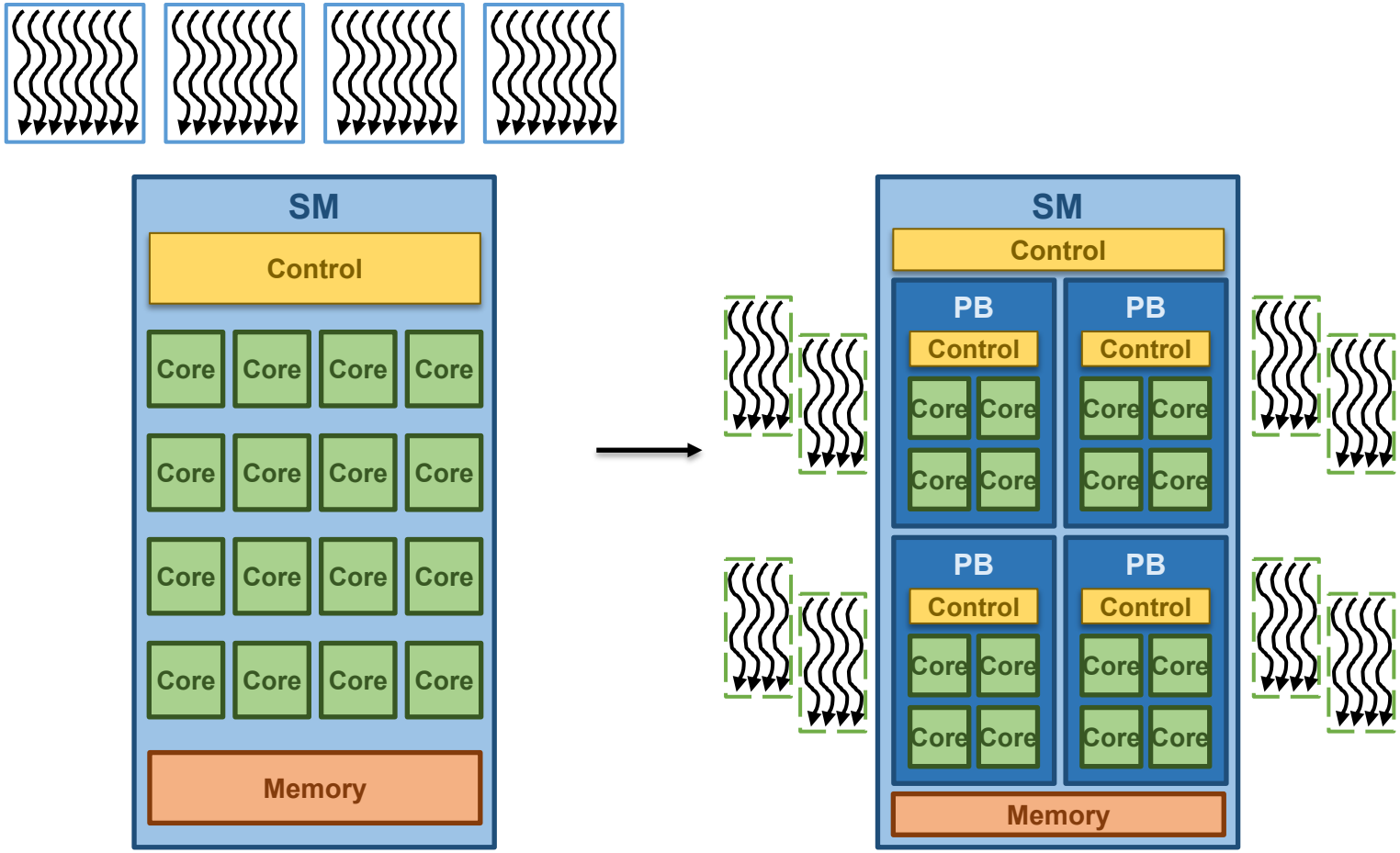
- Blocks assigned to an SM are further divided into **warps** which are the unit of scheduling
  - The SM cores are organized into **processing blocks**, with each processing block having its own warp scheduler to execute multiple warps concurrently



SM Scheduling

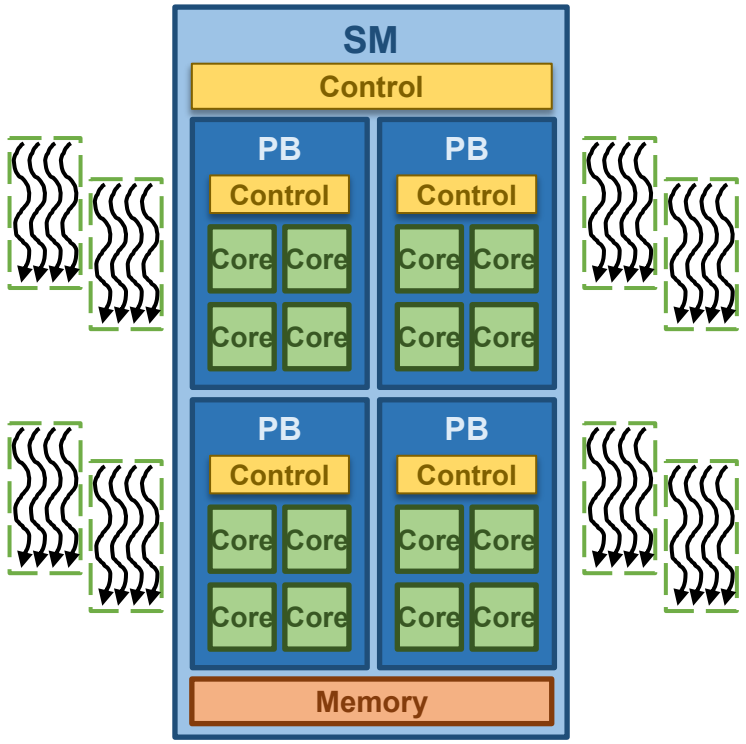


- Blocks assigned to an SM are further divided into **warps** which are the unit of scheduling
  - The SM cores are organized into **processing blocks**, with each processing block having its own warp scheduler to execute multiple warps concurrently



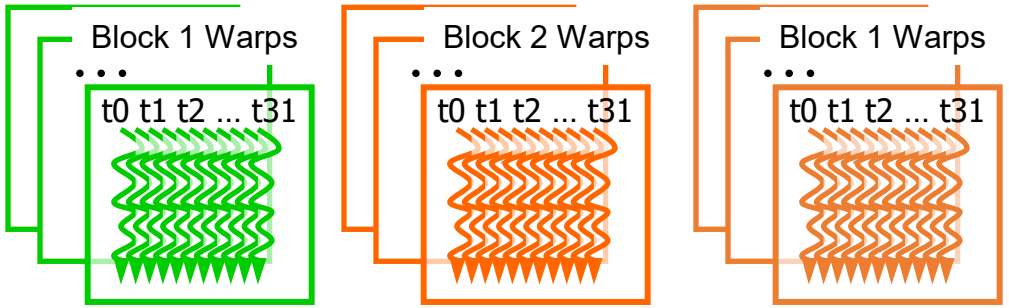
Warps

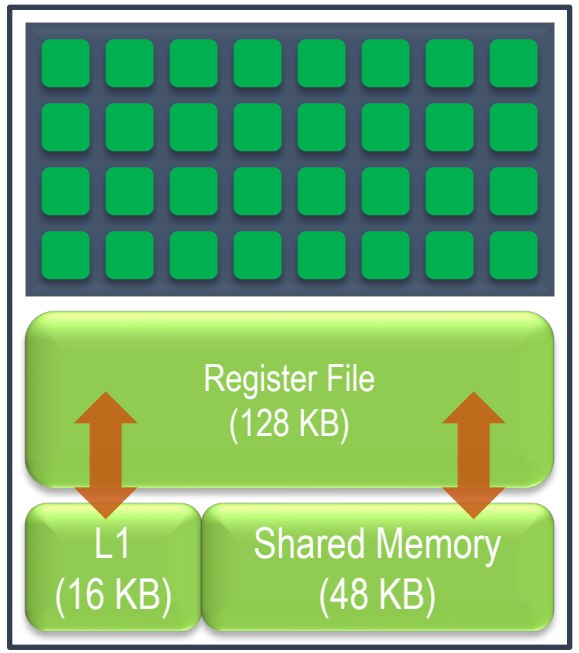
- The size of warps is device-specific, but has always been 32 threads to date
- Threads in a warp are scheduled together on a processing block and executed following the **SIMD** model
  - Single Instruction, Multiple Data
  - One instruction is fetched and executed by all the threads in the warp, each processing different data



Thread Scheduling

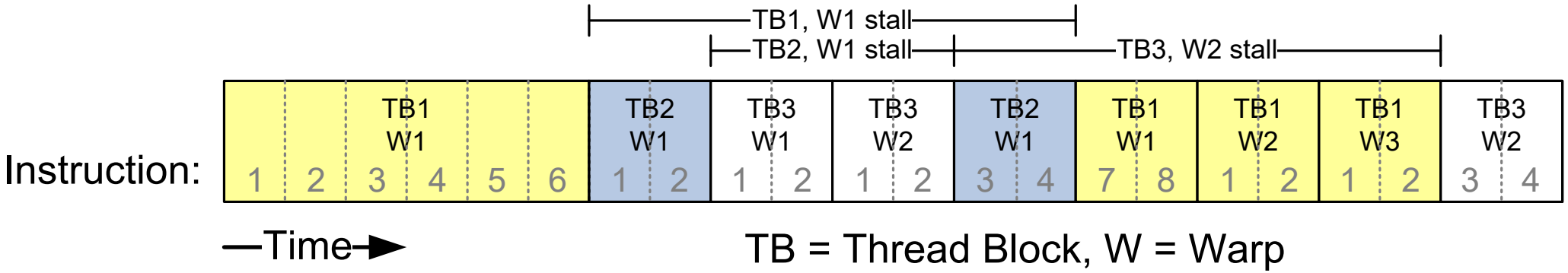
- Each block is executed as 32-thread warps
  - An implementation decision, not part of the CUDA programming model
  - Warps are divided based on their linearized thread index
    - Threads 0-31: warp 0
    - Threads 32-63: warp 1, etc.
  - Warps are scheduling units in SM
- If 3 blocks are assigned to an SM and each block has 256 threads, how many warps are there in an SM?
  - Each block is divided into 256/32 = 8 warps
  - 8 warps/blk * 3 blks = 24 warps





Thread Scheduling

- SM implements zero-overhead warp scheduling
  - Warps whose next instruction has its operands ready for consumption are eligible for execution
  - Eligible warps are selected for execution on a prioritized scheduling policy
  - **All threads in a warp execute the same instruction when selected**



Why SIMD?

- Advantage of SIMD:
  - Share the same instruction fetch/dispatch unit across multiple execution units (cores)
    - i.e., amortizes the cost of control across more execution units

Example GPU Architecture: H100

- Example: Hopper H100 GPU
  - SM has 128 FP32 cores
  - Organized into 4 processing blocks
  - Each processing block has 32 FP32 cores that share a dispatch unit
  - A warp is executed by one processing block

Why SIMD?

- Advantage of SIMD:
  - Share the same instruction fetch/dispatch unit across multiple execution units (cores)
    - i.e., amortizes the cost of control across more execution units

- Disadvantage of SIMD:
  - Different threads taking different execution paths result in **control divergence**
    - Warp does a pass over each unique execution path
    - In each pass, threads taking the path execute while others are disabled
  - The percentage of threads/cores enabled during SIMD execution is called the **SIMD efficiency**

Control Divergence Example

- **<u>Example 1:</u>** Threads take different branches of a conditional

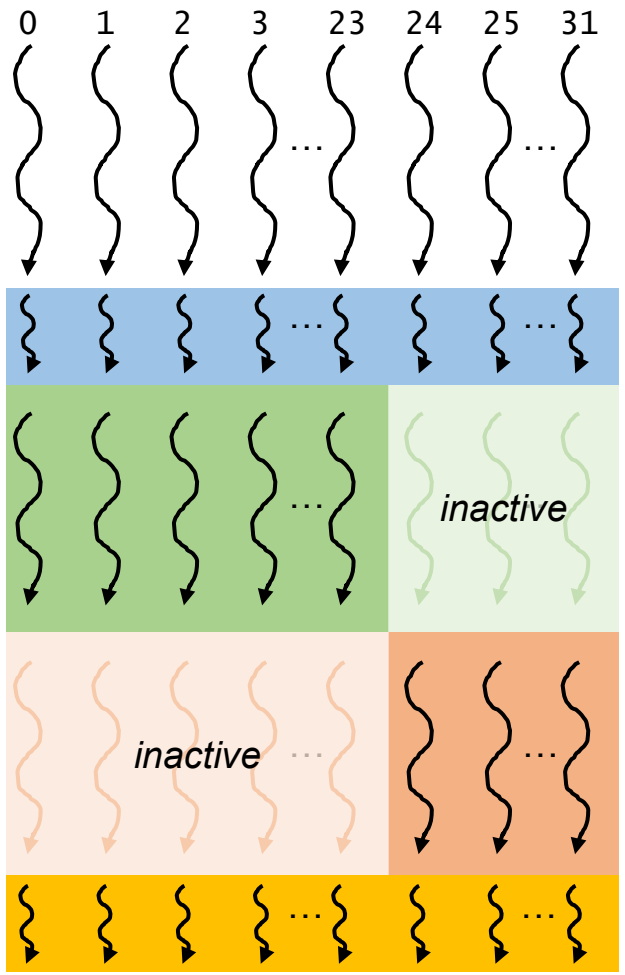
```
if(threadIdx.x < 24) {

    A

} else {

    B

}
C
```



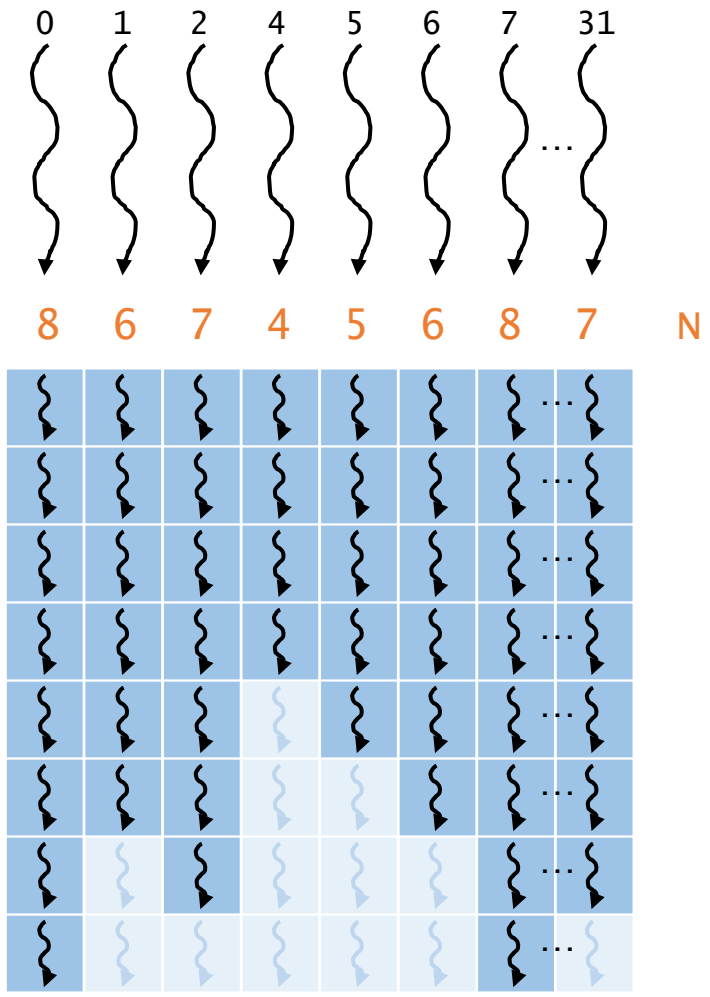
Control Divergence Example

- **<u>Example 2:</u>** Threads execute a different number of loop iterations

```
N = a[threadIdx.x];
for(i = 0; i < N; ++i) {

    A

}
```



Avoiding Branch Divergence

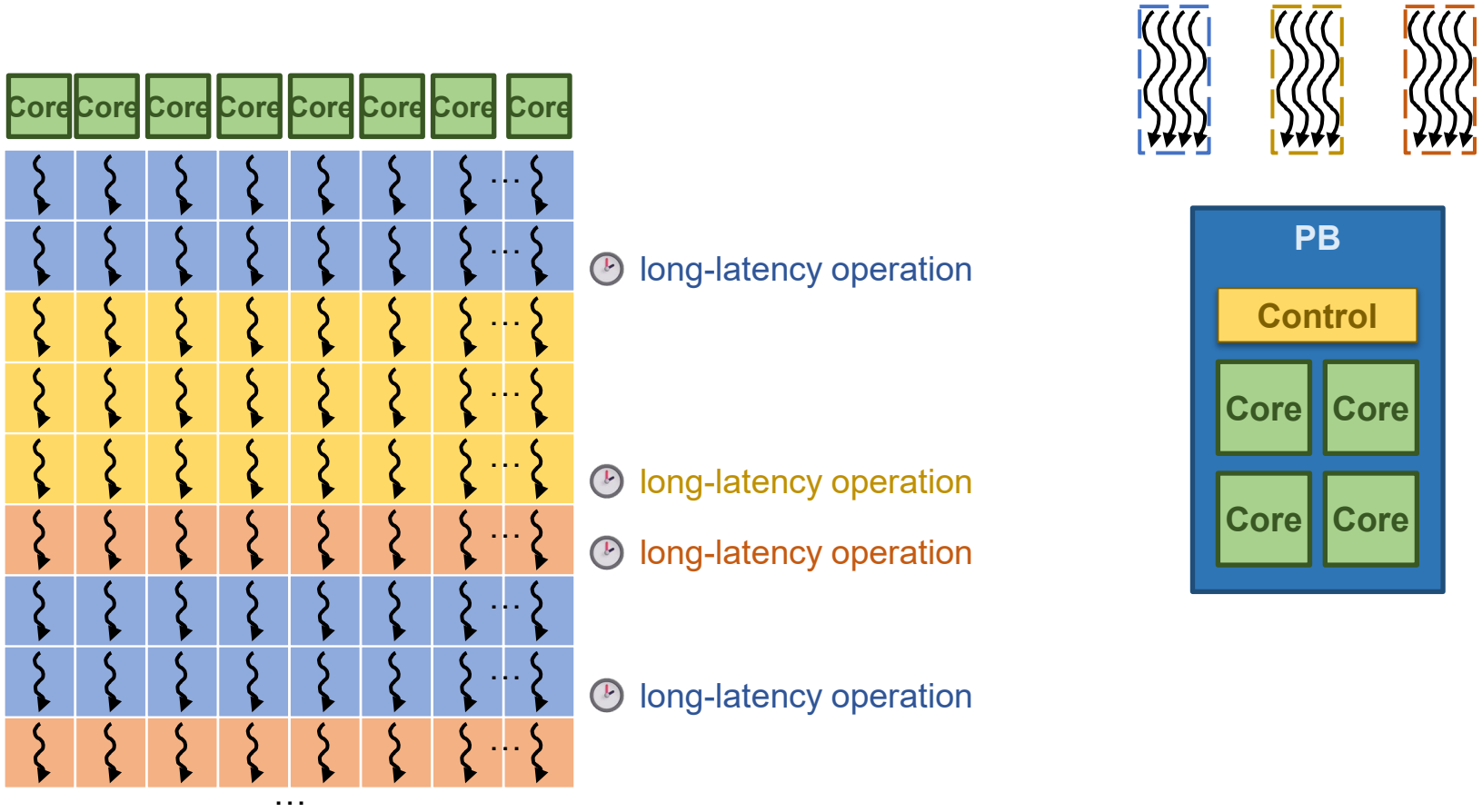
- Try to make branch granularity a multiple of warp size (remember, it may not always be 32!)

```
if (threadIdx.x / WARP_SIZE > 2) {
    // THEN path (lots of lines)
} else {
    // ELSE path (lots of lines)
}
```

- Still has two control paths
- But all threads in any warp follow only one path.

Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution



Latency Hiding

- When a warp needs to wait for a high latency operation, another warp that is ready is selected and scheduled for execution

- Many warps are needed so that there is sufficient work available to hide long latency operations
  - i.e., there is high chance of finding a warp that is ready to continue

- For this reason, an SM typically supports many more threads than the number of cores it has

SM Resources

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

- Max threads per SM is much higher than cores per SM

Occupancy

- The **occupancy** of an SM refers to the ratio of the warps or threads active on the SM to the maximum allowed

- In general, maximizing occupancy is desirable because it improves latency hiding
  - Common case, but possible to have cases where lower occupancy is desirable

Occupancy Constraints

GPU Name	K40	M40	P100	V100	A100	H100
Microarchitecture	Kepler	Maxwell	Pascal	Volta	Ampere	Hopper
Compute Capability	3.5	5.2	6.0	7.0	8.0	9.0
Number of SMs	15	24	56	80	108	132
Cores per SM	192	128	64	64	64	128
Max threads per SM	2048	2048	2048	2048	2048	2048
Max blocks per SM	16	32	32	32	32	32
Max threads per block	1024	1024	1024	1024	1024	1024
Registers per SM	64K	64K	64K	64K	64K	64K
Shared memory per SM	48KB	96KB	64KB	96KB	164KB	228KB

- Choice of threads per block constrains occupancy

Occupancy Example

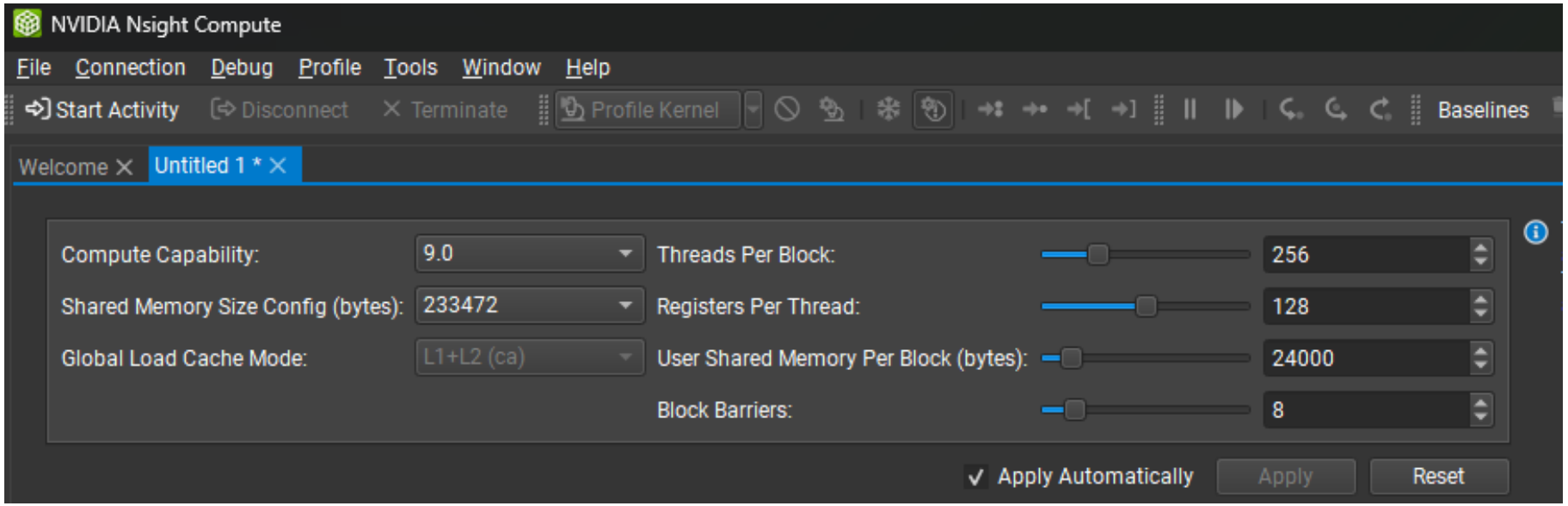
- Hopper H100 supports up to: 1024 threads/block
  2048 threads/SM, and **32 blocks/SM**
  - If **256 threads/block** are used, occupancy is maximized
    - (**2048 threads**)/(**256 threads/block**) = 8 blocks < **32 blocks**
  - If **32 threads/block** are used, the occupancy will be limited by the number of blocks allowed
    - (**2048 threads**)/(**32 threads/block**) = 64 blocks > **32 blocks**
  - If **768 threads/block** are used, occupancy will be limited because 2048 is not evenly divisible by 768
    - (**2048 threads**)/(**768 threads/block**) = 2 blocks
    - The remaining 512 cannot be used

Block Granularity Considerations

- For rgb2gray_kernel, should one use 8×8, 16×16 or 32×32 blocks? Assume that in the GPU used, each SM can take up to 1,536 threads and up to 8 blocks.
  - For 8×8, we have 64 threads per block. Each SM can take up to 1,536 threads, which is 1,536/64=24 blocks. But each SM can only take up to 8 Blocks, so only 512 threads (16 warps) go into each SM!
  - For 16×16, we have 256 threads per block. Each SM can take up to 1,536 threads (48 warps), which is 6 blocks (within the 8 block limit). Thus, we use the full thread capacity of an SM.
  - For 32×32, we have 1,024 threads per Block. Only one block can fit into an SM, using only 2/3 of the thread capacity of an SM.

Occupancy Calculator

- NVIDIA Nsight Compute provides an Occupancy Calculator that allows to compute the multiprocessor occupancy of a GPU for a given CUDA kernel.
  - It replaces the previously provided CUDA Occupancy Calculator spreadsheet.



Querying Available Resources

- Programmers can query the properties of a device to identify its available resources

```
cudaError_t cudaGetDeviceProperties(cudaDeviceProp* prop, int device)
```

- Example:

```
int devID = 0;
cudaDeviceProp devProp;
cudaGetDeviceProperties(&devProp, devID);
int maxThreadsPerBlock = devProp.maxThreadsPerBlock;
int maxThreadsPerMultiProcessor =
                devProp.maxThreadsPerMultiProcessor;
```

Any More Questions?

Things to Read:

Textbook chapter 4

CUDA PG: Programming Model

Things to Do:

Get Delta account

Setup GitHub

Submit Labs 0 and 1

Problem Solving

- Q: A particular CUDA device's streaming multiprocessor (SM) can take up to 1536 threads and up to 4 thread blocks. Which of the following block configurations could guarantee the SM be fully utilized?
  - 256 threads per block, 384 threads per block, 576 threads per block

- A:
  - 1536 / 256 = 6 thread blocks – too many for SM
  - 1536 / 384 = 4 thread blocks per SM – just the right number
  - 1536 / 576 = 3 thread blocks per SM – not enough to fully utilize the SM

Problem Solving

- Q: A 1D array of N floating point elements is to be processed in a one-element-per-thread fashion by a GPU. The target GPU has 8 SMs, each with 16 SPs. What is the best execution configuration for this kernel?

- A:
  - We do not know the max number of threads the SM can support.

Problem Solving

- Consider the following CUDA kernel:

```
__global__ void do_work(int q, int *A) {
      int result = 0;
      if (q < 5) result = threadIdx.x;
      A[threadIdx.x] = result;
 }
```

- Q: Is there is a control divergence in this code?

- A: No since the value of q is the same for ALL threads in the thread block. For a thread to diverge, its execution path must depend on something unique to the thread, e.g., on threadIdx.